

Montage Token (MTGX)

SMART CONTRACT

Security Audit

Performed on Contracts:

mtgx-erc20.sol

MD5 Hash: fc9a61af5d129b1c6feb0e42eb854049

Platform

ETH

hashlock.com.au

OCTOBER 2023

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Standardised Checks	9
Intended Smart Contract Functions	11
Function List	12
Code Quality	14
Audit Resources	14
Dependencies	14
Severity Definitions	15
Audit Findings	15
Centralisation	25
Conclusion	26
Our Methodology	27
Disclaimers	29
About Hashlock	30

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Montage Token team partnered with Hashlock to conduct a security audit of their ERC20 smart contract.. Hashlock manually and proactively reviewed the code in order to ensure the project was safe for launch, continued use and to give confidence to their community.

Project Context

Montage exists to ensure investors can trade crypto with confidence & to innovate a positive change through the adoption of digital currency. Their mission is to create a worry-free digital asset that levels the playing field for all traders, ensuring their investments are safe & secure.

Token Name: Montage Token

Contract Addresses (testnet): N/a

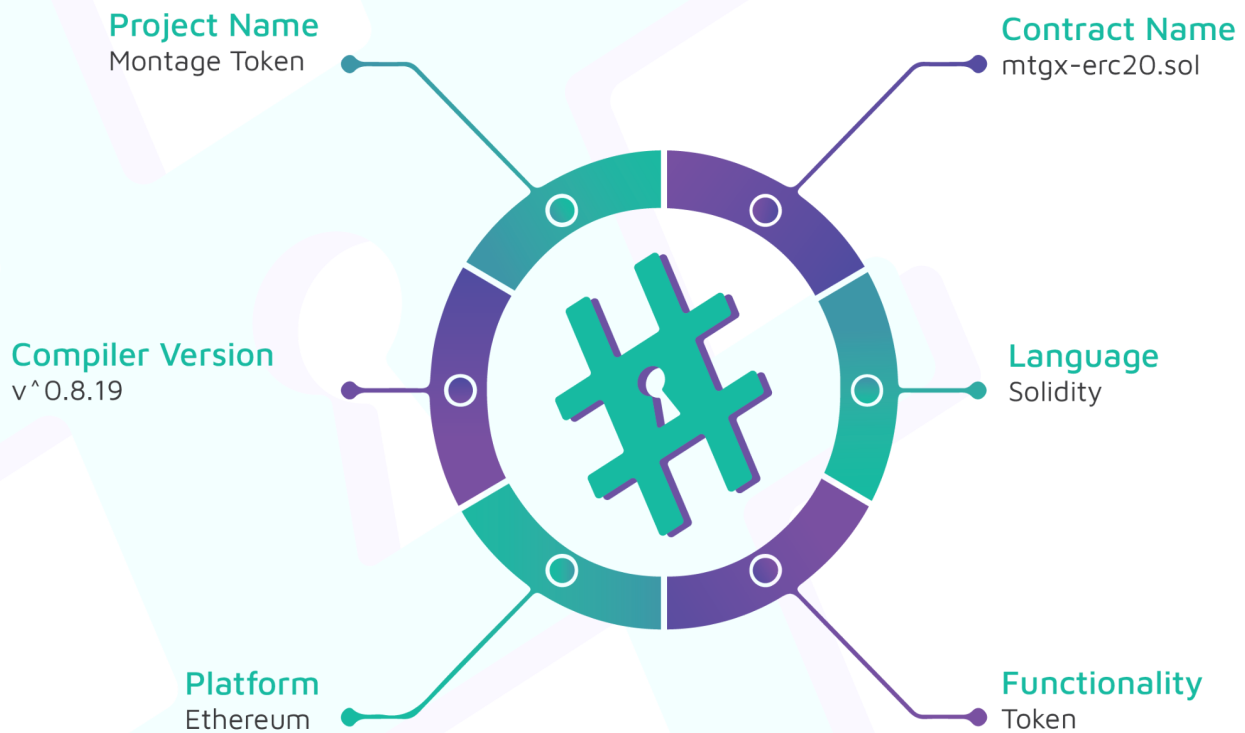
Compiler Version: 0.8.19+commit.7dd6d404

Logo:



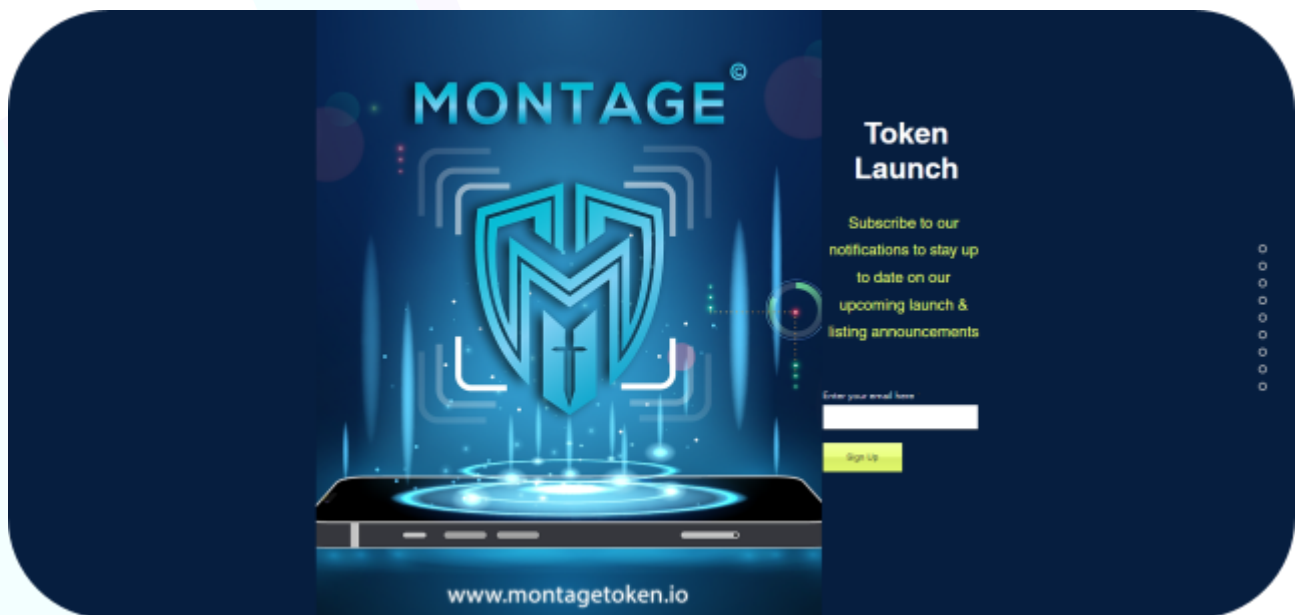
MONTAGE

Visualised Context:



#Hashlock.

Hashlock Pty Ltd

Platform Visuals:

Audit scope

Description	Project review and security analysis report for the Montage Token smart contract.
Platform	ETH / Solidity
Audit Date	Oct 2023
Contract 1	mtgx-erc20.sol
Contract 1 MD5 Hash	fc9a61af5d129b1c6feb0e42eb854049

Security Rating

After our audit and analysis, we found the smart contracts to be “**Hashlocked**”. The contracts all follow simple logic, with correct and detailed ordering. We have identified some varying levels of severity vulnerabilities that will need to be addressed before mainnet launch.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

Issues found:

2 High severity vulnerabilities

4 Medium severity vulnerabilities

3 Low severity vulnerabilities

3 Gas Optimisations

Caution: Hashlock's audit's do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity/compiler version stated	Passed
	Consistent pragma version across each contract	Passed
	Outdated Solidity Version	Passed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Passed
	Lack of check on input parameters	Passed
	Function input parameters check bypass	Passed
	Correct Access control	Reviewed
	Built in emergency features	Reviewed
	Correct event logs	Passed
	Human/contract checks bypass	Reviewed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Reviewed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Reviewed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Reviewed
	Use keywords/functions to be deprecated	Passed
	Unused code	Passed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed

	High consumption 'storage' storage	Passed
	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Passed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Initial Audit Result: Failed

Revised Audit Result: Passed

Intended Smart Contract Functions

Claimed Behaviour	Result
mtgx-erc20.sol <u>Features:</u> • Transfers between users • Swap to ETH on fee on transfer • Add liquidity • Allows admin to: ◦ Block/unblock accounts ◦ Include/exclude from fees, wallet max limits, set fees on sell/buy, sets wallets, ◦ Sets uniswap router ◦ Claim launch tokens ◦ Claim ETH overflows	Reviewed

Function List

mtgx-erc20.sol contracts

Functions

	Functions	Type	Observation	Conclusion
1	activateTrading	write, onlyOwner	Passed	No Issue
2	_setAutomatedMarketMakerPair	private	Passed	No Issue
3	allowTradingWhenDisabled	write, onlyOwner	Passed	No Issue
4	blockAccount	write, onlyOwner	Passed	No Issue
5	unblockAccount	write, onlyOwner	Passed	No Issue
6	excludeFromFees	write,only Owner	Passed	No Issue
7	excludeFromMaxWalletLimit	write, onlyOwner	Passed	No Issue
8	setWallets	write, onlyOwner	Passed	No Issue
9	setFeesOnBuy	write, onlyOwner	Passed	No Issue
10	setFeesOnSell	write, onlyOwner	Passed	No Issue
11	setUniswapRouter	write, onlyOwner	Passed	No Issue
12	setMaxWalletAmount	write, onlyOwner	Passed	No Issue
13	setMinimumTokensBeforeSwap	write, onlyOwner	Passed	No Issue
14	claimLaunchTokens	write, onlyOwner	Passed	No Issue
15	claimETHOverflow	write, onlyOwner	Passed	No Issue
16	getBuyFees	read	Passed	No Issue

17	getSellFees	read	Passed	No Issue
18	_transfer	private	Passed	Reviewed
19	_adjustTaxes	private	Passed	Reviewed
20	_setCustomSellTaxPeriod	private	Passed	No Issue
21	_setCustomBuyTaxPeriod	private	Passed	No Issue
22	_swapAndTransfer	private	Passed	Reviewed
23	_swapTokensForETH	private	Passed	Reviewed
24	_addLiquidity	private	Passed	No Issue

Code Quality

This audit scope involves the solidity smart contracts of the Montage project, as outlined in the Audit Scope section. All contracts, libraries and interfaces follow standard best practices and avoid unnecessary complexity that increases the likelihood of exploitation.

All code is well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are very helpful in understanding the overall architecture of the protocol.

We were also provided with project documentation and links to assist with our understanding of the project's architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

High

[H-01] Interactions with Uniswap can be sandwiched, resulting in significantly worse execution

Description

The `_swapTokensForETH` function uses 1 wei as the minimum amount of ETH out. This means that every swap is exposed to potentially unlimited slippage.

Vulnerability Details

The Uniswap V2 router has the function `swapExactTokensForETHSupportingFeeOnTransferTokens` with the following parameters:

Name	Type	Description
amountIn	uint	The amount of input tokens to send.
amountOutMin	uint	The minimum amount of output tokens that must be received for the transaction not to revert.
path	address[]	An array of token addresses. path.length must be ≥ 2 . Pools for each consecutive pair of addresses must exist and have liquidity.
to	address	Recipient of the ETH.
deadline	uint	Unix timestamp after which the transaction will revert.

The `_swapTokensForETH` function sets `amountOutMin = 1` when calling this function, meaning that it is possible for the swap to be sandwich attacked by an MEV bot.

```
function _swapTokensForETH(uint256 tokenAmount) private {
    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = uniswapV2Router.WETH();
    _approve(address(this), address(uniswapV2Router), tokenAmount);
    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokenAmount,
        1, // accept any amount of ETH
        path,
        address(this),
        block.timestamp
    );
}
```


Impact

Swaps can be sandwich attacked by a MEV bot, resulting in significantly worse execution on these swaps.

Recommendation

Create a TWAP oracle for the MTGX/ETH Uniswap v2 pool.

Have a `slippageTolerance` state variable that allows a small amount of slippage (0.5% is Uniswap frontend's default).

Use the TWAP oracle price and the `slippageTolerance` to calculate the minimum amount of ETH to receive for each swap.

Status

Resolved

[H-02] Fee swaps front-run user swaps, resulting in high slippage

Description

Whenever a user sells MTGX tokens into the Uniswap V2 pool, their trade could be front-run by the automated fee swap. This results in very high slippage and bad UX for the user.

Vulnerability Details

Whenever a user sells MTGX tokens into the Uniswap V2 pool and there are enough tokens inside the MTGX token contract to call `_swapAndTransfer`, the automated swap front-runs the user, resulting in a worse price for the user.

The amount of slippage that the user incurs is based on three factors:

1. The amount of liquidity in the Uniswap V2 pool.
2. The size of the automated swap (at least 2.5 million tokens are sold).
3. The size of the user's swap.

Uniswap's frontend allows users to set their own slippage tolerance on their swap (default is 0.5%). If the slippage tolerance is too small, the user's swap will fail, resulting in bad UX. If the user sets their slippage tolerance high so that the swap succeeds, they will have to accept a significantly worse price on their trade.

Impact

If the user triggers the automated fee swap, then they either have to accept a significantly worse price on their trade, or are unable to execute the swap successfully due to slippage protection.

Recommendation

Instead of having fee swaps triggered by user interactions with the Uniswap V2 pool, perform the swap in its own transaction on a regular schedule (such as once a week). You can automate this with an off-chain script.

Status

Acknowledged

Developer comments;

We refer to this as the swap and liquefy functions that treat the distribution of "token taxes" to their various destinations. Hashlock has identified that in order to the distributions to take place the contract front-runs a token sale by a token holder. The contract is designed this way, to operate as trustless and automated manner so that there is no need or participation required from the MONTAGE team to sell any collected tokens. From an optics perspective swap and liquefy is more transparent for the users trusting that the contract will manage token taxes accordingly. If we have the team accrue tokens to sell at a random time, in our opinion, can cause more damage as holders will begin to question sold amounts, team selling, frequencies etc. In our opinion its best to keep the contract as the sole controller of tax distribution. Perhaps a better setting for the swap and liquefy threshold would be 0 tokens to reduce the sale size.

This being said, if the MONTAGE team would prefer to control the selling of their own token for taxes manually, then we are more than happy to update the code to accommodate.

Brewlabs recommendation: Leave code as is and acknowledge finding

Medium

[M-01] `_transfer` - Blocked accounts that are allowed to trade tokens when trading is disabled can bypass checks

This finding is related to [M-02]

Description

An account that is allowed to trade when trading is disabled can bypass blocks and max wallet limits, since these checks are only done if both the `from` and `to` address are not allowed to trade when trading is disabled.

Vulnerability Details

The following code snippet checks if an account is allowed to transfer tokens.

```

if (!_isAllowedToTradeWhenDisabled[from] && !_isAllowedToTradeWhenDisabled[to]) {
    require(isTradingEnabled, "Montage: Trading is currently disabled.");
    require(!_isBlocked[to], "Montage: Account is blocked");
    require(!_isBlocked[from], "Montage: Account is blocked");

    if (!_isExcludedFromMaxWalletLimit[to]) {
        require((balanceOf(to) + amount) <= maxWalletAmount, "Montage: Expected wallet
amount exceeds the maxWalletAmount.");
    }
}

```

Since all the checks are contained within the `if` statement, these checks are only performed if both the `from` and `to` addresses aren't allowed to trade when trading is disabled.

Impact

If either the `from` or `to` addresses are allowed to trade when trading is disabled, then the following can happen:

- Both `from` and `to` addresses can send and receive tokens even if either of them are blocked
- Max wallet limits can be bypassed.

Recommendation

Put the checks on `_isBlocked` outside of the `if`-statement.

Status

Acknowledged

[M-02] `_transfer` - Tokens can be transferred when trading is disabled if either sender or recipient are allowed to trade while trading is disabled

This finding is related to [M-01]

Description

Due to the condition inside the `if` condition, tokens can be transferred if either the `from` or `to` address are allowed to trade when trading is disabled.

Vulnerability Details

```

if (!_isAllowedToTradeWhenDisabled[from] && !_isAllowedToTradeWhenDisabled[to]) {
    require(isTradingEnabled, "Montage: Trading is currently disabled.");
    require(!_isBlocked[to], "Montage: Account is blocked");
    require(!_isBlocked[from], "Montage: Account is blocked");

    if (!_isExcludedFromMaxWalletLimit[to]) {
        require((balanceOf(to) + amount) <= maxWalletAmount, "Montage: Expected wallet
amount exceeds the maxWalletAmount.");
    }
}

```

Since `&&` is used, both the `from` and `to` addresses must not be allowed to trade when trading is disabled for the checks inside the `if` statement to trigger.

Impact

Tokens can be transferred if either the `from` or `to` address are allowed to trade when trading is disabled.

Recommendation

Use `||` instead of `&&` inside the condition of the `if`-statement.

Status

Acknowledged

[M-03] `_adjustTaxes` - 100% liquidity fee after launch is never applied

Description

Since `block.timestamp` is compared with `_launchBlockNumber`, 100% liquidity fees are never applied.

Vulnerability Details

A 100% liquidity fee is supposed to be applied on the first 5 blocks after trading is activated. However, since `block.timestamp` is used instead of `block.number`, this condition will never be satisfied:

```
if (isBuy) {
  if (_isLaunched && block.timestamp - _launchBlockNumber <= 5) {
    _targetAFee = 100000;
  } else {
    _targetAFee = _base.targetAFeeOnBuy;
    _targetBFee = _base.targetBFeeOnBuy;
    _targetCFee = _base.targetCFeeOnBuy;
    _burnFee = _base.burnFeeOnBuy;
  }
}
```

Impact

The 100% liquidity fee on buys after launch will never be applied.

Recommendation

Change `block.timestamp` to `block.number`.

Status

Acknowledged

[M-04] `_transfer` - Transfers are disabled when trading is disabled

Description

Due to how the if-statements and `require` statements are implemented, users who are not allowed to trade when trading is disabled are also not allowed to transfer their tokens.

Vulnerability Details

When trading is disabled, users who hold Montage tokens also cannot transfer their tokens to any address that isn't allowed to trade when trading is disabled. This is due to the if-statement below in `_transfer`:

```
if (!_isAllowedToTradeWhenDisabled[from] && !_isAllowedToTradeWhenDisabled[to]) {
  require(isTradingEnabled, "Montage: Trading is currently disabled.");
  require(!_isBlocked[to], "Montage: Account is blocked");
  require(!_isBlocked[from], "Montage: Account is blocked");

  if (!_isExcludedFromMaxWalletLimit[to]) {
    require((balanceOf(to) + amount) <= maxWalletAmount, "Montage: Expected wallet
amount exceeds the maxWalletAmount.");
  }
}
```

The `require(isTradingEnabled, "Montage: Trading is currently disabled.");` check reverts if trading is disabled. Users who aren't allowed to trade when trading is disabled also cannot transfer their tokens.

Recommendation

Remove the `require(isTradingEnabled, "Montage: Trading is currently disabled.");` line.

The if-statement condition checks for `isTradingEnabled`, which is sufficient to only allow automated swaps to execute only if trading is enabled.

```
if (
  isTradingEnabled &&
  canSwap &&
  !_swapping &&
  _totalFee > 0 &&
  automatedMarketMakerPairs[to]
) {
  _swapping = true;
  _swapAndTransfer();
  _swapping = false;
}
```

If the intended behavior is to prevent users from trading with the Uniswap V2 pool when `isTradingEnabled = false`, then the following line can be added to `_transfer`:

```
require(
  isTradingEnabled ||
  !automatedMarketMakerPairs[from] ||
  !automatedMarketMakerPairs[to],
```

```
"Montage: Trading is disabled"
);
```

Status

Acknowledged

Low

[L-01] `_transfer` - Function should transfer fee before swapping

Description

Inside the `_transfer` function, the fee swap is performed before the user sends the fee. This means that there will always be tokens leftover in the token contract, even after an automated fee swap.

Recommendation

`_transfer` should transfer the fee before calling `_swapAndTransfer`.

Status

Acknowledged

[L-02] Initial token supply is inconsistent with white paper

Description

The initial token supply stated on the Montage Token website and implemented on the smart contract is 10 billion tokens:

```
uint256 public initialSupply = 10000000000 * (10**18);
```

This is inconsistent with the Montage Token white paper, which says that the total supply of Montage Token is 1 trillion tokens.

Recommendation

Depending on what's intended, change the `initialSupply` and Montage Token website, or change the total supply stated in the white paper.

Status

Acknowledged

[L-03] `_transfer` - Max wallet limit check uses wrong `amount` `value`

Description

The actual amount that is transferred to the `to` address is less than or equal to the `amount` input parameter, since fees are deducted.

```

if (!_isAllowedToTradeWhenDisabled[from] && !_isAllowedToTradeWhenDisabled[to]) {
    ...
    if (!_isExcludedFromMaxWalletLimit[to]) {
        require((balanceOf(to) + amount) <= maxWalletAmount, "Montage: Expected wallet
amount exceeds the maxWalletAmount.");
    }
}
...
if (takeFee && _totalFee > 0) {
    uint256 fee = (amount * _totalFee) / 100000;
    amount = amount - fee;
    super._transfer(from, address(this), fee);
}
super._transfer(from, to, amount);

```

As shown above, the `amount` value in `require((balanceOf(to) + amount) <= maxWalletAmount, ...)` uses the pre-fee value. This overestimates the amount of tokens that is about to be sent to the user.

Recommendation

Perform the max wallet limit check after the fee has been deducted from `amount`.

Status

Acknowledged

Gas

[G-01] `_adjustTaxes` - Function does not need to manipulate storage

Description

The `_adjustTaxes` function accesses and makes changes to the following state variables:

- `_targetAFee`
- `_targetBFee`
- `_targetCFee`
- `_burnFee`

These values are only calculated and used in one transaction. Afterwards, they are reset. There is no need to keep their values in storage after a transaction. Hence, storing these variables in storage is unnecessary.

Recommendation

Change `_adjustTaxes` to a view function that returns `(_totalFee, _targetAFee, _targetBFee, _targetCFee, _burnFee)` which should all be memory variables.

Status

Resolved

[G-02] `_swapAndTransfer` - Unnecessary `_totalFee` assignment

Description

The following code snippet is inside `_swapAndTransfer`:

```
function _swapAndTransfer() private {  
    uint256 contractBalance = balanceOf(address(this));  
    uint256 initialETHBalance = address(this).balance;  
    uint32 _totalFeePrior = _totalFee;  
    ...  
    _totalFee = _totalFeePrior;  
}
```

The last line is unnecessary, since the value of `_totalFeePrior` does not change throughout the function.

Recommendation

Remove the `_totalFee = _totalFeePrior` line.

Status

Resolved

[G-03] `_adjustTaxes` only needs to be called if a fee is incurred

Description

The `_adjustTaxes` function gets called in each token transfer. This spends unnecessary gas, as the tax values only need to be adjusted if the transaction is taking a fee from the user.

Recommendation

Only call `_adjustTaxes` if it's known that a fee will be applied.

Status

Resolved

Centralisation

The Montage token values safe and trusted investment however, there exists multiple controls which may stop the free flow of token transfers between users or other contracts. As a result, this was rated closer towards the centralised indicator.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Montage Token seems to have a sound code base, pending the resolution of our findings. Overall, most of the code is correctly ordered and follows industry best practices.

Since possible test cases can be unlimited, we provide no such guarantee of future outcomes. We have used both static tools and manual line-by-line analysis to minimise the probability of contract failure and notify users of potential rug vectors so they can bear the risks on themselves.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.

